

Zelo — Handoff Técnico Completo

Documento gerado em: 31 de julho de 2026 **Propósito:** permitir que outro desenvolvedor assuma o projeto sem acesso ao histórico de decisões.

1. Visão Geral

Objetivo

O Zelo é uma plataforma de intermediação entre famílias que precisam de serviços residenciais e profissionais autônomos que os oferecem. Não é uma agência de empregos nem uma empresa prestadora — é um espaço de conexão, verificação e reputação.

Público-alvo

Famílias em Pelotas/RS que buscam contratar diaristas (limpeza residencial), babás, cuidadoras de idosos e motoristas particulares. Do outro lado, profissionais autônomas que querem visibilidade e um canal de confiança com clientes novos, sem pagar mensalidade.

Modelo de negócio

Intermediação pura. O Zelo:

- **não** é empregador;
- **não** processa pagamentos (cliente e profissional acertam isso fora da plataforma);
- **não** define preços (cada profissional define os próprios valores);
- **não** garante qualidade ou resultado do serviço.

O valor do produto está em três pilares: **verificação de identidade** (documento + selfie conferidos manualmente), **reputação por avaliação dupla e simultânea** (evita retaliação) e **referências de trabalho confirmadas por telefone** (selo bronze/prata/ouro).

Fluxo principal

1. Profissional se cadastra, aceita termos e declarações, envia documento de identidade e selfie.
2. Admin confere manualmente e aprova pelo painel.
3. Perfil fica visível na busca automaticamente (coluna gerada no banco, ninguém “publica” manualmente).
4. Cliente busca por categoria/bairro/turno, ordenado por completude de perfil.

5. Cliente contrata — isso abre o WhatsApp da profissional automaticamente e avisa o admin por e-mail.
6. Serviço acontece fora da plataforma.
7. Ambos avaliam um ao outro; as avaliações só ficam públicas quando os dois avaliaram (ou 14 dias se passaram).

2. Arquitetura

Stack

Camada	Tecnologia
Frontend	React 18 + Vite 5
Roteamento	react-router-dom v6
Backend	Supabase (Postgres + Auth + Storage + Edge Functions)
Hospedagem do site	Netlify
E-mail transacional	Resend (SMTP + API)
Domínio	zeloemcasa.com.br (registro.br)

Dependências (`package.json`)

```
"dependencies": {
  "@supabase/supabase-js": "^2.45.0",
  "react": "^18.3.1",
  "react-dom": "^18.3.1",
  "react-router-dom": "^6.26.0"
},
"devDependencies": {
  "@vitejs/plugin-react": "^4.3.1",
  "vite": "^5.4.0"
}
```

Sem framework de CSS (Tailwind, etc.) — estilo é CSS puro em `src/styles/global.css`, mais estilos inline no JSX para casos pontuais. Sem biblioteca de gerenciamento de estado (Redux, Zustand) — tudo é `useState` / `useEffect` local ou Context (`useAuth`).

Estrutura de pastas

```
src/
  components/      — componentes reutilizáveis (Header, Rodape, cartões, modais)
```

```
hooks/          - useAuth.jsx (único hook customizado do projeto)
lib/            - api.js (toda comunicação com Supabase) e supabase.js (client)
pages/          - uma página por rota
styles/         - global.css (único arquivo de estilo)
App.jsx         - definição de rotas e proteção de acesso
main.jsx        - bootstrap do React

supabase/
  01 a 27_*.sql - migrações numeradas, sequenciais, idempotentes quando possível
  functions/    - três Edge Functions (Deno)
  CONFIGURACAO_SMTP.md
```

Padrão adotado: `api.js` como única porta de entrada ao banco

Nenhum componente chama `supabase.from(...)` diretamente (exceto casos raros documentados). Toda query vive em `src/lib/api.js`, exportada como função nomeada. Isso foi decisão deliberada para centralizar a lógica de queries — especialmente porque o PostgREST (API REST do Supabase) tem uma limitação importante documentada abaixo.

Padrão de código: comentários explicam POR QUÊ, não O QUÊ

O código tem comentários extensos em pontos de decisão não óbvia — principalmente onde um bug real já aconteceu e foi corrigido. Isso é intencional: o histórico de bugs (seção 17) mostra que vários problemas eram sutis e silenciosos (RLS sem policy, corrida de estado assíncrono). Os comentários existem para que a próxima pessoa não reintroduza o mesmo erro.

Limitação técnica importante: PostgREST e joins N:N

Descoberta cedo no desenvolvimento, custou várias rodadas de debug: selects aninhados do PostgREST através de tabelas de junção com chave composta (ex.: `profissional_categorias -> categorias`) falham **silenciosamente** — retornam vazio, sem erro. A solução adotada em todo o projeto: nunca usar select aninhado para relações N:N; sempre fazer consultas separadas e montar o objeto em JavaScript. Isso está em quase toda função de `api.js` que lida com categorias, bairros, serviços.

3. Funcionalidades Implementadas

3.1 Autenticação e cadastro

Finalidade: permitir que clientes e profissionais criem conta, com dados obrigatórios diferentes por papel.

Como funciona: `Cadastro.jsx` tem duas telas — primeiro a escolha do papel (cliente/profissional), depois o formulário. Profissional passa por três aceites adicionais: Termo de Consentimento (agora comum aos dois papéis), Declaração de Antecedentes, e duas declarações simples (veracidade das informações, aptidão legal).

O cadastro usa `supabase.auth.signUp()` com os dados extras (nome, papel, cidade, bairro, aceites) dentro de `options.data` — isso vira `raw_user_meta_data` no Postgres. Um **trigger no banco** (`criar_perfil_ao_registrar` , migração 19/20/24/25) lê esses metadados e cria as linhas em `perfis` e `profissionais` automaticamente, **sem depender de sessão ativa**.

Por que via trigger e não insert direto do frontend: quando a confirmação de e-mail está ligada, `signUp()` não cria sessão até o clique no link do e-mail — um insert feito pelo navegador nesse meio-tempo falharia (não há `auth.uid()`). O trigger roda com privilégio de sistema, então funciona nos dois cenários.

Arquivos: `src/pages/Cadastro.jsx` , `src/hooks/useAuth.jsx` (função `cadastrar`) , `supabase/19_perfil_via_trigger.sql` , `20_declaracao_antecedentes.sql` , `24_termo_todos_usuarios.sql` , `25_data_verificacao.sql` .

Componentes usados: `TermoConsentimento.jsx` , `DeclaracaoAntecedentes.jsx` .

3.2 Login e recuperação de senha

Finalidade: autenticação padrão + fluxo de “esqueci minha senha”.

Como funciona: `Login.jsx` chama `entrar()` e depois **espera** o perfil chegar via `useEffect` , reagindo à mudança de estado do contexto de `auth` — não tenta decidir o destino no mesmo tick do clique (isso já causou bug, ver seção 17).

Recuperação: `RecuperarSenha.jsx` chama `supabase.auth.resetPasswordForEmail()` , sempre com a mesma mensagem de sucesso (não revela se o e-mail existe). `NovaSenha.jsx` recebe a sessão temporária do link e chama `supabase.auth.updateUser({ password })` .

Arquivos: `src/pages/Login.jsx` , `RecuperarSenha.jsx` , `NovaSenha.jsx` , `src/hooks/useAuth.jsx` .

Dependência de configuração: exige `/nova-senha` cadastrado em Authentication → URL Configuration → Redirect URLs no Supabase.

3.3 Busca de profissionais

Finalidade: cliente encontra profissionais por categoria, bairro e turno.

Como funciona: `Busca.jsx` — formulário com chips de categoria, seletor de bairro, data, turno. A função `buscarProfissionais` em `api.js` filtra por categoria/bairro via tabelas de junção (consultas separadas, nunca `select aninhado`), depois por compatibilidade de turno (quem atende “integral” aparece em buscas por “manhã” ou “tarde”), e por fim ordena por **pontuação de completude de perfil** (ver 3.9).

Exige login (rota protegida em `App.jsx`) — antes era pública, mudou por decisão de produto.

Arquivos: `src/pages/Busca.jsx` , `src/lib/api.js` (`buscarProfissionais`).

3.4 Perfil público da profissional

Finalidade: vitrine completa — o que ela faz, valores, avaliações, selo.

Como funciona: `PerfilProfissional.jsx` busca tudo via `obterProfissional()`, que faz múltiplas consultas paralelas (categorias, bairros, serviços — nunca aninhado). Mostra: foto, categorias, bairros atendidos, valores (meio turno / diária / km, dependendo da categoria), serviços específicos (se Limpeza Residencial), selo de identidade confirmada (clicável, com modal explicativo), selo de referências (bronze/prata/ouro, sem revelar conteúdo), avaliações, e o aviso de intermediação.

Contratação: botão “Contratar” verifica se há referência aprovada disponível (`contarReferenciasAprovadas` — só a contagem, nunca o conteúdo). Se houver, abre o `DisclaimerReferencia` antes de prosseguir. Ao confirmar, cria o booking, abre o WhatsApp da profissional automaticamente (com mensagem pré-escrita), e dispara e-mail com a referência (se aplicável) via Edge Function.

Arquivos: `src/pages/PerfilProfissional.jsx`, componentes `SeloReferencias.jsx`, `SeloVerificacao.jsx`, `AvisoIntermediacao.jsx`, `DisclaimerReferencia.jsx`.

3.5 Painel do cliente

Finalidade: ver contratações ativas e histórico, avaliar, conversar.

Arquivos: `src/pages/PainelCliente.jsx`, `src/components/Chat.jsx`, `FormAvaliacao.jsx`.

3.6 Painel da profissional

Finalidade: central de operação da profissional — pedidos, perfil, agenda, verificação, ganhos, avaliações.

Estrutura por abas: Pedidos, Meu perfil, Agenda, Verificação, Ganhos, Avaliações.

Verificação é o ponto mais sensível: upload de identidade e selfie (bucket privado), com aviso de que o uso é exclusivamente para conferência. **Importante:** identidade e selfie condicionam só a **visibilidade na busca**, não o acesso ao painel — isso foi implementado, revertido e reimplementado durante o desenvolvimento (ver seção 15).

Arquivos: `src/pages/PainelProfissional.jsx` (arquivo grande, contém também o subcomponente `Verificacao`), `src/components/Agenda.jsx`, `EditarPerfil.jsx`, `Referencias.jsx`.

3.7 Avaliações duplas e simultâneas

Finalidade: reputação confiável sem retaliação.

Como funciona: cada lado avalia o outro após o serviço concluído. Nenhuma avaliação aparece até **ambos** terem avaliado — ou até 14 dias se passarem (função `publicar_avaliacoes_vencidas`, que precisa ser agendada via cron externo, **isso ainda não foi configurado**, ver Pendências). Cada avaliação tem um campo de comentário público (lido só por pessoas do mesmo lado — clientes leem comentários de clientes, profissionais leem de profissionais) e um campo de **feedback privado**, lido só pelo destinatário, também só após a publicação simultânea.

Bug histórico corrigido: a view `minhas_avaliacoes_recebidas` foi criada com `security_invoker = true`, o que fazia o RLS negar a própria pessoa que deveria ler — a funcionalidade nunca funcionou até ser corrigida na migração 09.

Arquivos: `src/components/FormAvaliacao.jsx`, `Avaliacoes.jsx`, `FeedbackPrivado.jsx`, `TrustScore.jsx`.

3.8 Referências de trabalho e selos

Finalidade: dar credibilidade adicional além da avaliação — contato de clientes anteriores, confirmado pelo admin por telefone.

Como funciona (estado ATUAL, pós-reversão): profissional cadastra até 3 referências (nome + telefone de clientes anteriores). Admin liga para confirmar e aprova/rejeita. Cada aprovada conta para o selo (1=bronze, 2=prata, 3=ouro), calculado por trigger no banco.

O contato NUNCA aparece no perfil público. Só o selo genérico é visível. Quando o cliente contrata, se há referência aprovada, vê um **disclaimer legal** (implicações de uso indevido de dado de terceiro, LGPD) e, ao aceitar, recebe por **e-mail** o primeiro nome + telefone de uma referência. Isso é feito por uma Edge Function (`enviar-referencia`) que revalida tudo no servidor — nunca confia no frontend.

Admin pode bloquear uma referência já aprovada a qualquer momento (se a pessoa citada reclamar) — isso a remove da contagem do selo e impede novos envios, sem apagar o histórico.

Histórico de vaivém nesta funcionalidade: o projeto passou por três estados — (1) só admin via o contato, (2) contato público no perfil para qualquer visitante, (3) estado atual: contato só por e-mail, após contratação e disclaimer. Ver seção 15 para a timeline completa.

Arquivos: `src/components/Referencias.jsx` (visão da profissional), `SeloReferencias.jsx`, `DisclaimerReferencia.jsx`, `src/pages/PainelAdmin.jsx` (seção de bloqueio), `supabase/17_referencias_selos.sql`, `26_referencias_publicas.sql` (revertida pela 27), `27_referencias_privadas_bloqueio.sql`, `supabase/functions/enviar-referencia/`.

3.9 Pontuação de completude de perfil

Finalidade: incentivar perfis completos, ordenando-os primeiro na busca.

Critérios (1 ponto cada, máx. 5): foto de perfil, descrição preenchida, valores definidos, agenda preenchida, referência aprovada (e não bloqueada). Calculado por trigger no banco (`recalcular_pontuacao_perfil`), disparado por mudanças em `profissionais`, `perfis.foto_url`, `disponibilidade` e `referencias_trabalho`.

Descoberta durante a implementação: o app lia `foto_url` em vários lugares mas **não existia nenhuma tela de upload de foto** — foi preciso criar `enviarFotoPerfil` e o campo correspondente antes de a pontuação fazer sentido.

Arquivos: `supabase/22_pontuacao_perfil.sql`, `src/lib/api.js` (`buscarProfissionais` ordena por `pontuacao_perfil desc`), `src/components/EditarPerfil.jsx` (barra de progresso "Perfil X/5 completo").

3.10 Notificações por WhatsApp e e-mail

WhatsApp (pessoal, não Business): ao contratar, o app monta um link `wa.me` com o telefone

cadastrado pela profissional e abre automaticamente no navegador do cliente — antes mesmo da profissional aceitar. Decisão explícita: velocidade de contato acima de intermediação formal via API paga da Meta.

E-mail ao admin: Edge Function `notificar-cadastro`, acionada por até quatro Database Webhooks (INSERT em `profissionais`, `verificacoes`, `referencias_trabalho`; UPDATE em `verificacoes` para o e-mail de “parabéns” à profissional quando aprovada).

E-mail de parabéns à profissional: dentro da mesma função, dispara quando a aprovação de um documento **completa o par** (identidade + selfie ambas aprovadas) — não a cada aprovação individual.

Arquivos: `supabase/functions/notificar-booking/` (pedido novo → e-mail ao admin, WhatsApp Business foi removido dessa função), `notificar-cadastro/` (cadastro/documento/referência pendente + parabéns).

3.11 LGPD e conformidade legal

Ver seção 12 completa. Resumo: Termos de Uso e Política de Privacidade em páginas próprias, termo de consentimento no cadastro (ambos os papéis), declaração de antecedentes (só profissional), consentimento específico de dado sensível no momento do upload de selfie, área “Minha conta e privacidade” (exportar dados, encerrar conta), selo de verificação com modal explicativo, rodapé com links permanentes, disclaimer legal específico para divulgação de referência de terceiro.

4. Fluxo do Cliente

1. **Cadastro:** escolhe “Preciso contratar” → preenche nome, e-mail, senha, cidade, bairro → aceita Termo de Consentimento → conta criada (trigger cria o perfil).
2. **Login:** e-mail + senha. Se confirmação de e-mail estiver ligada, precisa clicar no link antes.
3. **Busca:** categoria (chips), bairro, data, turno → resultados ordenados por completude de perfil.
4. **Filtros:** categoria, bairro, turno (com compatibilidade: quem atende “integral” aparece em buscas por “manhã” ou “tarde”).
5. **Contratação:** no perfil da profissional, botão Contratar. Se há referência aprovada, mostra disclaimer antes. Confirma → cria booking → abre WhatsApp da profissional → (se aplicável) recebe e-mail com referência.
6. **Avaliação:** após o serviço, avalia no painel (`PainelCliente.jsx`) — nota + comentário público + mensagem privada opcional. Só publica quando a profissional também avaliar.
7. **Favoritos:** tabela `favoritos` existe no schema, mas **não há UI implementada** — funcionalidade pendente (roadmap declarado, não é bug).
8. **Mensagens:** chat por booking, via Supabase Realtime (`src/components/Chat.jsx`).
9. **Pagamentos:** não existem na plataforma — acontece fora, por decisão de modelo de negócio (reduz risco regulatório de intermediação financeira).

10. **Perfil/conta:** em "Minha conta e privacidade" — exportar dados (JSON), encerrar conta (com confirmação por digitação "EXCLUIR"), revogar consentimento (equivale a encerrar).
-

5. Fluxo do Profissional

1. **Cadastro:** escolhe "Quero oferecer serviços" → nome, e-mail, senha, cidade, bairro → aceita Termo de Consentimento + Declaração de Antecedentes + duas declarações simples (veracidade, aptidão legal) → conta criada.
 2. **Edição de perfil:** aba "Meu perfil" — foto, descrição, idade, telefone, WhatsApp, categorias (chips), serviços específicos (se Limpeza Residencial), bairros atendidos (ou "TODOS"), valores (meio turno / diária / km rodado, conforme categoria).
 3. **Documentos:** aba "Verificação" — upload de identidade (RG/CNH) e selfie (bucket privado `documentos-verificacao`). **Antecedentes está temporariamente desativado** como exigência (decisão de negócio, migração 16) — o campo de upload não existe mais nessa aba, mas a Declaração de Antecedentes (autodeclarada, sem verificação) continua no cadastro.
 4. **Selfie:** campo com `capture="user"`, abre câmera frontal no celular. Consentimento específico de dado sensível é mostrado no momento do envio (não mais no cadastro geral).
 5. **Antecedentes:** desativado como exigência de documento. Existe autodeclaração no cadastro (checkbox), sem verificação pela plataforma — texto explícito disso no próprio checkbox.
 6. **Disponibilidade:** aba "Agenda" — `disponibilidade` (dia da semana + turno) e `dias_bloqueados` (datas específicas indisponíveis).
 7. **Recebimento de contatos:** quando um cliente contrata, o pedido aparece na aba "Pedidos". A profissional recebe o contato do cliente via WhatsApp (aberto automaticamente do lado do cliente) — a profissional não precisa fazer nada para isso acontecer.
 8. **Avaliações:** avalia clientes após serviço concluído, mesma lógica dupla/simultânea do lado do cliente.
-

6. Painele Administrativo

O que existe:

- Fila de verificação (identidade + selfie pendentes/em análise) — aprovar ou solicitar correção. Documentos abrem via **URL assinada de 60 segundos** (nunca URL pública).
- Fila de referências pendentes — aprovar (após ligação de confirmação) ou marcar como não confirmada.
- Lista de referências já aprovadas — com opção de **bloquear** (com motivo opcional) ou desbloquear.

O que falta implementar:

- Dashboard com métricas gerais (quantas profissionais ativas, quantos bookings no mês, etc.) — não existe nenhuma visão agregada hoje.
- Gestão de usuários (banir, suspender conta) além do que já existe via SQL manual.
- Qualquer relatório ou exportação de dados agregados.
- Interface para editar categorias/bairros/serviços disponíveis (hoje só via SQL Editor).

Arquivo: `src/pages/PainelAdmin.jsx`.

7. Banco de Dados

Tipos enum

```
create type user_role      as enum ('cliente', 'profissional', 'admin');
create type verif_status   as enum ('pendente', 'em_analise', 'aprovado', 'rejeitado');
create type verif_metodo   as enum ('manual', 'idwall', 'unico', 'serpro');
create type booking_status as enum ('solicitado', 'confirmado', 'concluido', 'cancelado', ' ');
create type review_lado    as enum ('cliente_avalua_prof', 'prof_avalua_cliente');
```

Tabelas

idades

Cidades atendidas pelo Zelo. Hoje só Pelotas ativa (mais 3 previstas como semente, inativas).

bairros

Bairros por cidade. 7 cadastrados em Pelotas + “Outro / não listado” (criada na migração 11, para o cliente que não encontra o próprio bairro).

categorias

Tipos de serviço: Limpeza Residencial (renomeada de “Faxineira” na migração 23), Babá, Cuidadora de idosos, Motorista Particular (migração 21). RLS: leitura pública, sem escrita pelo app.

perfis

1:1 com `auth.users`. Colunas principais: `id`, `role` (`user_role`), `nome`, `foto_url`, `cidade_id`, `bairro_id`, `termo_aceito`, `termo_aceito_em`, `termo-versao` (migração 24). **Telefone não fica aqui** — foi movido para `contatos` (migração 05) porque a policy de leitura pública de perfis expunha telefone de qualquer usuário para qualquer outro logado.

RLS: select público (using (true)); insert/update só do próprio (id = auth.uid()), mas na prática o insert é feito pelo trigger de cadastro, não pelo frontend.

profissionais

Estende perfis quando role = 'profissional'. Colunas acumuladas ao longo do projeto:

descricao, idade, experiencia, especialidades, valor_meio_turno (renomeado de valor_hora, migração 12), valor_diaria, valor_km (migração 21), atende_todos_bairros (migração 11), identidade_status, antecedentes_status, selfie_status (migração 10), identidade_verificada_em (migração 25), consentimento_verificacao/em/versao (migração 14), declaracao_antecedentes/em/versao (migração 20), declarou_info_verdadeiras, declarou_apto_legalmente (migração 25), servico_outro (migração 23), selo (migração 17), pontuacao_perfil (migração 22), visivel (coluna GERADA, ver abaixo).

Coluna visivel — o coração da regra de negócio:

```
generated always as (  
    identidade_status = 'aprovado' and selfie_status = 'aprovado'  
) stored
```

Passou por várias versões (identidade+antecedentes+selfie → identidade+selfie, quando antecedentes foi desativado como exigência — migração 16). **Atenção:** toda vez que essa coluna é alterada, a policy prof_select_visiveis (que depende dela) precisa ser dropada e recriada — Postgres não permite alterar coluna da qual uma policy depende. Isso já causou um bug real (busca retornando vazia sem erro nenhum — ver seção 17).

RLS: select por visivel = true or id = auth.uid() or auth_role() = 'admin'. Insert/update pela própria ou pelo admin (prof_update_admin , necessária para o admin conseguir aprovar verificação).

profissional_categorias, profissional_bairros, profissional_servicos

Tabelas de junção N:N. RLS: leitura pública, escrita só do dono. **Achado na auditoria (migração 09):** essas tabelas tinham grant de escrita **sem RLS ligado** — qualquer usuário logado podia editar a agenda ou categorias de qualquer profissional. Corrigido.

servicos_disponiveis

Lista fixa de serviços específicos por categoria (ex.: dentro de Limpeza Residencial: “Limpeza geral”, “Lavar roupas” etc.). Migração 23. RLS: leitura pública.

verificacoes

Um registro por documento enviado (tipo : identidade/selfie/antecedentes; status ; documento_path no bucket privado). RLS: select da própria ou admin; insert da própria (verif_insert_own , adicionada depois — faltava no desenho original); update só admin.

Trigger importante: sincronizar_status_verificacao — quando uma verificação muda de status,

sincroniza automaticamente `identidade_status / selfie_status / antecedentes_status` em `profissionais`. Antes desse trigger, o admin tinha que fazer dois updates manuais (um em `verificacoes`, outro em `profissionais`), e isso **já dessincronizou de fato** em produção (bug real, ver seção 17).

`disponibilidade, dias_bloqueados`

Agenda da profissional (dia da semana + turno; datas específicas bloqueadas). RLS corrigido na auditoria (mesma falha de RLS desligado com grant de escrita).

`bookings`

Contratações. `status (booking_status), categoria_id, bairro_id, data_servico, turno, valor_combinado`. RLS: select pelas duas partes; insert pelo cliente; update pelas duas partes (sem restrição de coluna — **pendência de segurança conhecida**, ver seção 17).

`avaliacoes`

`lado (review_lado), nota, comentario (público, segmentado por lado), comentario_privado` (só o destinatário, só após publicação), `publicada_em` (null até ambos avaliarem ou 14 dias passarem). RLS complexa: select segmentado por `auth_role()` e `lado`; **sem policy de UPDATE** (proposital — avaliação não se edita).

View `minhas_avaliacoes_recebidas`: lê o feedback privado. Recriada na migração 09 **sem `security_invoker`** — com ele, o RLS da tabela base negava a leitura à própria pessoa que deveria ler (bug real, funcionalidade nunca operou até a correção).

Trigger/função `publicar_avaliacoes_vencidas()`: existe no banco, mas **não está agendada** — precisa de `pg_cron` ou uma função agendada externa. Pendência crítica.

`mensagens`

Chat por booking. Realtime ativo. RLS: só as duas partes do booking.

`favoritos`

Existe no schema (`alternarFavorito, listarFavoritos` em `api.js`), mas **sem tela nenhuma** — roadmap declarado.

`contatos`

Telefone/WhatsApp separado de `perfis` (migração 05, por vazamento de RLS). RLS: cada um lê o próprio; leitura por booking confirmado (`contatos_select_booking_confirmado`) e por booking recém-criado (`contatos_select_ao_contratar`, migração 18, para o WhatsApp abrir na hora do clique em Contratar); admin lê tudo.

`notificacoes_log`

Log de tentativas de envio de e-mail/WhatsApp (sucesso ou falha), para diagnosticar silenciosamente sem depender de “a profissional reclamar que não recebeu”. RLS: só admin lê.

referencias_trabalho

`nome_referencia`, `telefone`, `status` (`verif_status`), `bloqueada` (boolean, migração 27), `bloqueada_em`, `bloqueada_motivo`. Limite de 3 por profissional, **imposto por trigger** (`checar_limite_referencias`), não só na tela.

RLS: própria profissional e admin leem tudo; **não há mais policy pública** (a migração 26 tinha criado uma, revertida pela 27). Uma função `contar_referencias_aprovadas(uuid)` com `security definer` devolve **só a contagem**, nunca o conteúdo, para o frontend decidir se mostra o disclaimer.

divulgacoes_referencia

Log de quando/para quem uma referência foi enviada por e-mail, se o disclaimer foi aceito. Migração 27. RLS: só admin lê; só a Edge Function (service role) escreve.

Funções/triggers mais importantes (visão consolidada)

Função	Dispara em	Faz
<code>criar_perfil_ao_registrar</code>	AFTER INSERT em <code>auth.users</code>	Cria perfis e profissionais a partir de <code>raw_user_meta_data</code>
<code>sincronizar_status_verificacao</code>	AFTER UPDATE em <code>verificacoes</code>	Sincroniza status em profissionais
<code>recalcular_selo</code>	AFTER INSERT/UPDATE/DELETE em <code>referencias_trabalho</code>	Recalcula bronze/prata/ouro
<code>recalcular_pontuacao_perfil</code>	Várias tabelas	Recalcula pontuação de completude (0-5)
<code>checar_limite_referencias</code>	BEFORE INSERT em <code>referencias_trabalho</code>	Impede mais de 3 referências
<code>auth_role()</code>	Chamada dentro de policies	Retorna o <code>role</code> do usuário logado — precisa de <code>security definer</code> , senão RLS bloqueia a própria leitura (bug histórico)
<code>contar_referencias_aprovadas(uuid)</code>	RPC chamada pelo frontend	Só a contagem, nunca o conteúdo

8. Autenticação

Supabase Auth (GoTrue), e-mail + senha. Client configurado com `persistSession: true`, `autoRefreshToken: true` (`src/lib/supabase.js`).

Confirmação de e-mail: SMTP próprio via Resend configurado (domínio `zeloemcasa.com.br` verificado). **Status da confirmação de e-mail no momento deste handoff não está confirmado nesta documentação** — verificar em Authentication → Settings no painel do Supabase antes de assumir qualquer coisa.

Papéis: `cliente`, `profissional`, `admin` — um usuário não acumula papéis (decisão de produto, não há mecanismo técnico que impeça uma pessoa de ter duas contas com e-mails diferentes, porém).

Proteção de rotas: `App.jsx` tem um componente `Protegida` que verifica sessão e, opcionalmente, papel específico. Se `perfil` não carrega (usuário órfão — conta em `auth.users` sem linha em `perfis`), mostra tela de erro com botão Sair, em vez de tela em branco infinita (bug corrigido).

Bugs de corrida já corrigidos (ver seção 17): login não redirecionando por race condition entre `signInWithPassword` e leitura de `perfil`; cadastro mostrando tela de "conta não encontrada" pela mesma razão. Solução final: `Login.jsx` e o próprio `useAuth` reagem a mudanças de estado via `useEffect`, nunca tentam prever timing do SDK.

9. Storage

Dois buckets no Supabase Storage:

`documentos-verificacao` (privado)

Identidade e selfie. Caminho: `{profissional_id}/{tipo}-{timestamp}.{ext}`. Acesso só via **URL assinada de 60 segundos**, nunca `getPublicUrl()`. Extensão do arquivo é sanitizada (`/^[a-z0-9]{1,5}$` / , senão vira `.bin`) — o nome do arquivo vem do usuário e não é confiável.

`fotos-perfil` (público)

Foto de perfil da profissional. `upsert: true` (permite trocar a foto). Cache-busting via query string `?t=timestamp`, porque o nome do arquivo não muda entre uploads.

Retenção de documentos sensíveis: o texto legal do app **não promete mais** exclusão automática dos documentos após aprovação (isso foi removido intencionalmente a pedido do usuário) — mas também não há rotina automática de exclusão implementada. Se isso for reintroduzido como promessa legal, precisa vir junto de automação real.

10. APIs

Supabase REST (PostgREST)

Toda leitura/escrita de dados do app passa por aqui, via `@supabase/supabase-js`. Autenticação: JWT da sessão do usuário (anon key + token). RLS decide o que cada papel vê.

Supabase Auth API

`signUp`, `signInWithPassword`, `resetPasswordForEmail`, `updateUser`, `signOut`, `getSession`, `onAuthStateChange`.

Supabase Realtime

Usado só na tabela `mensagens` (chat).

Edge Functions (Deno, três no total)

Função	Chamada por	Finalidade
<code>notificar-booking</code>	Database Webhook (INSERT em <code>bookings</code>)	E-mail ao admin avisando pedido novo
<code>notificar-cadastro</code>	Database Webhooks (INSERT em <code>profissionais / verificacoes / referencias_trabalho</code> ; UPDATE em <code>verificacoes</code>)	E-mail ao admin (cadastro/documento/referência pendente) + e-mail de parabéns à profissional quando aprovada
<code>enviar-referencia</code>	Chamada direta do frontend (<code>supabase.functions.invoke</code>)	Envia por e-mail o contato de referência ao cliente, após disclaimer aceito

Todas usam `SUPABASE_SERVICE_ROLE_KEY` (bypassa RLS deliberadamente, sempre revalidando manualmente o que é necessário) e `RESEND_API_KEY`. `notificar-cadastro` e `notificar-booking` usam `ADMIN_EMAIL`.

Configuração necessária em cada função: desligar “Verify JWT with legacy secret” nas Settings da função — os Database Webhooks não enviam esse token, e com a verificação ligada a função recebe 401 sempre.

API externa: Resend

`https://api.resend.com/emails`, autenticação via Bearer token (a mesma API key usada no SMTP de Auth). Remetente: `contato@zeloemcasa.com.br` (domínio verificado).

11. Segurança

- **RLS em todas as tabelas** que contêm dado de usuário — nenhuma tabela de dado sensível fica aberta por padrão.

- **Grants explícitos** — o projeto foi criado com "Automatically expose new tables" desligado; toda tabela nova precisa de grant manual além do RLS. Isso já causou confusão real (RLS correto, mas sem grant = "permission denied" sem explicação — ver seção 17).
- **security definer COM set search_path = public** em toda função que precisa contornar RLS de propósito (ex.: `auth_role()` , `contar_referencias_aprovadas`), para evitar sequestro de schema.
- **URLs assinadas de curta duração (60s)** para documentos sensíveis, nunca URL pública.
- **Revogação de TRUNCATE/REFERENCES/TRIGGER** de `anon / authenticated` (auditoria, migração 09) — TRUNCATE ignora RLS e tinha sido concedido por engano.
- **Headers de segurança no Netlify** (`netlify.toml`): CSP restrita ao necessário (Supabase + Google Fonts), X-Frame-Options DENY, nosniff, Referrer-Policy, Permissions-Policy.
- **Edge Functions nunca confiam no frontend** para decisões de autorização — ex.: `enviar-referencia` extrai o cliente real do token, não de um parâmetro enviado pelo app.
- **Sanitização de nome de arquivo** em todo upload.

Pendência de segurança conhecida: `bookings` tem policy de UPDATE para as duas partes **sem restrição de coluna** — nada impede uma das partes de alterar `valor_combinado` depois do fato. Baixo risco hoje (pagamento é fora da plataforma), mas vale trigger de colunas permitidas por papel antes de escalar.

12. LGPD

Implementado

- Termos de Uso e Política de Privacidade em páginas públicas (`/termos` , `/privacidade`), linguagem simples, versionadas por constante (`VERSAO_TERMOS_USO` , `VERSAO_PRIVACIDADE`).
- Termo de Consentimento e Ciência do Usuário — obrigatório para cliente E profissional, texto cobrindo base legal, papel de intermediação, responsabilidade do profissional pelas próprias informações, alcance limitado da verificação, responsabilidade do cliente na decisão de contratar. Registro de aceite com data, hora e versão (`perfis.termo_aceito/em/versao`).
- Declaração de Antecedentes Criminais — autodeclaração, sem verificação pela plataforma, texto explícito disso. Registro próprio (`profissionais.declaracao_antecedentes/em/versao`).
- Consentimento específico para dado sensível (selfie) — movido do cadastro geral para o momento do upload, no painel da profissional, por ser dado biométrico (art. 11 LGPD) que exige consentimento destacado.
- Área "Minha conta e privacidade" (`/minha-conta`): exportação de dados em JSON (direito de acesso/portabilidade), encerramento de conta com confirmação por digitação, explicação sobre revogação de consentimento.

- Selo de verificação clicável com modal explicando o alcance limitado ("conferência na data da análise", não garantia permanente).
- Rodapé com links permanentes para os documentos legais e contato.
- Disclaimer legal específico para divulgação de contato de terceiro (referência de trabalho), citando a LGPD e responsabilidades civil/criminal.
- Log de divulgação de dado de terceiro (`divulgacoes_referencia`).

Faltando / conhecido como incompleto

- **Exclusão de conta não remove o login do Supabase Auth** — só o perfil. O registro de autenticação fica órfão até o admin apagar manualmente. Isso está descrito na própria tela de "Minha conta", mas não é resolvido automaticamente.
- **Registro de IP no consentimento** (pedido em um dos documentos anexados) — não implementado. Exigiria capturar IP no servidor (Edge Function), não no navegador.
- **Banner de cookies** — não implementado, mas a Política de Privacidade já declara que só cookies essenciais são usados, então tecnicamente pode não ser obrigatório. Revisar com jurídico.
- **Versionamento com exigência de novo aceite** quando os termos mudam — a infraestrutura de versão existe (`termo-versao` é gravado), mas **não há mecanismo que force re-aceite** quando a versão atual do app diverge da versão que a pessoa aceitou. Isso precisa ser construído: comparar `termo-versao` salvo contra a constante atual do componente, e bloquear uso até novo aceite.
- **Exclusão automática de documentos de verificação após aprovação** — texto legal não promete mais isso (removido a pedido), e não há rotina implementada. Se reintroduzir a promessa, precisa vir com automação.
- **Consultoria jurídica formal** — todo o texto legal foi escrito com base em boas práticas e nos documentos fornecidos pelo usuário, mas não houve revisão por advogado real antes deste handoff.

13. Interface

Identidade visual

Paleta: verde-sálvia escuro (`--sage-900`) como cor de texto/destaque principal, coral (`--coral`) como cor de ação, tons de creme/papel (`--cream` , `--paper`) como fundo. Tipografia serifada (Fraunces) para títulos, sans-serif (Inter) para corpo de texto.

Logo: "Z" estilizado com casa e folha, em verde e coral — arquivo `public/zelo-logo.jpeg` .

Componentes reutilizáveis

- `Header.jsx` — logo (link contextual: painel se logado, home se não), navegação condicional por papel.

- `Rodape.jsx` — links legais e contato.
- `SeloReferencias.jsx`, `SeloVerificacao.jsx` — badges visuais.
- `TrustScore.jsx` — nota média + contagem de avaliações.
- `AvisoIntermediacao.jsx`, `TermoConsentimento.jsx`, `DeclaracaoAntecedentes.jsx`, `DisclaimerReferencia.jsx` — blocos de texto legal padronizados.

Telas existentes (rotas)

Rota	Arquivo	Papel exigido
<code>/</code>	<code>Home.jsx</code>	nenhum
<code>/entrar</code>	<code>Login.jsx</code>	nenhum
<code>/cadastrar</code>	<code>Cadastro.jsx</code>	nenhum
<code>/recuperar-senha</code>	<code>RecuperarSenha.jsx</code>	nenhum
<code>/nova-senha</code>	<code>NovaSenha.jsx</code>	sessão temporária do link
<code>/buscar</code>	<code>Busca.jsx</code>	qualquer logado
<code>/profissional/:id</code>	<code>PerfilProfissional.jsx</code>	qualquer logado
<code>/painel</code>	<code>PainelCliente.jsx</code>	cliente
<code>/painel-profissional</code>	<code>PainelProfissional.jsx</code>	profissional
<code>/admin</code>	<code>PainelAdmin.jsx</code>	admin
<code>/minha-conta</code>	<code>MinhaConta.jsx</code>	qualquer logado
<code>/termos</code>	<code>TermosUso.jsx</code>	nenhum
<code>/privacidade</code>	<code>Privacidade.jsx</code>	nenhum

Todas as rotas exceto Home são carregadas via `React.lazy` (code splitting) — decisão para manter o bundle inicial pequeno no celular.

14. Arquivos Importantes

Arquivo	Para que serve
<code>src/lib/api.js</code>	Toda comunicação com o banco — arquivo

	mais importante do projeto, ~1090 linhas
<code>src/hooks/useAuth.jsx</code>	Contexto de autenticação, único hook customizado
<code>src/App.jsx</code>	Rotas e proteção de acesso
<code>src/lib/supabase.js</code>	Inicialização do client Supabase
<code>supabase/01_schema.sql</code>	Schema base — tabelas, enums, primeira versão de tudo
<code>supabase/02_rls.sql</code>	RLS original
<code>supabase/09_auditoria.sql</code>	Correções de segurança encontradas em auditoria pré-lançamento
<code>supabase/19_perfil_via_trigger.sql</code>	Trigger de criação de perfil — crítico para o cadastro funcionar com confirmação de e-mail
<code>supabase/27_referencias_privadas_bloqueio.sql</code>	Estado final (até este handoff) da política de referências
<code>src/pages/PainelProfissional.jsx</code>	Maior arquivo de página, contém o subcomponente <code>Verificacao</code>
<code>README.md</code>	Documentação de setup original do projeto

15. Linha do Tempo de Alterações

Ordem cronológica aproximada, por tema (não por data exata de cada commit):

1. Schema inicial, RLS, seed de Pelotas/categorias/bairros, app React básico, deploy Netlify.
2. Debugging extenso de produção: URL do Supabase duplicada, grants faltando, policies de INSERT ausentes, selects aninhados N:N falhando silenciosamente, `auth_role()` sem `security definer`, roteamento de papel incorreto.
3. Auditoria pré-lançamento completa: RLS desligado em 4 tabelas com grant de escrita, TRUNCATE concedido indevidamente, view de feedback privado nunca funcionou, busca pública quebrada, trigger de sincronização de verificação criado.
4. Feedback privado nas avaliações (liberação simultânea).
5. Turno integral (manhã+tarde) como opção própria de agenda e cobrança.
6. Melhorias de cadastro: escolha de papel em destaque, "outros bairros"/"atende todos", valor por meio turno em vez de valor/hora, home simplificada (só login/cadastro).
7. Vários bugs de corrida em login/cadastro (perfil não carregava a tempo) — corrigidos com padrão

de reagir a `useEffect` em vez de prever timing.

8. WhatsApp automático via Meta Cloud API — **tentado, abandonado** por complexidade de configuração (aprovação de templates, conta Business). Substituído por WhatsApp pessoal (link `wa.me` direto) + e-mail ao admin.
9. Domínio próprio (`zeloemcasa.com.br`) registrado e configurado no Netlify; SMTP próprio via Resend configurado e verificado.
10. Selos bronze/prata/ouro por referências de trabalho — implementados com aprovação do admin por telefone.
11. Auditoria e correção de textos: remoção da palavra “verificada” de vários lugares (troca por linguagem mais factual).
12. Categoria Motorista Particular + valor por km.
13. Categoria Faxineira renomeada para Limpeza Residencial + lista de serviços específicos.
14. Estrutura LGPD completa: Termos, Privacidade, Minha Conta, selo clicável, disclaimer.
15. Termo de Consentimento reescrito e expandido para cliente + profissional (antes só profissional).
16. Notificações por e-mail ao admin (cadastro, documento, referência) — Edge Function + Database Webhooks, configurado com dificuldade real (extensões `pg_net` / `supabase_functions` , tipo de webhook).
17. **Selfie obrigatória para acessar o painel** — implementado, depois **revertido** a pedido do usuário: selfie/identidade voltaram a condicionar só a visibilidade na busca, não o acesso.
18. E-mail de parabéns à profissional quando aprovada (par completo identidade+selfie).
19. **Referências de trabalho tornadas públicas no perfil** (migração 26) — depois **revertidas** (migração 27): voltaram a ser privadas, com entrega só por e-mail após contratação + disclaimer legal + capacidade do admin de bloquear.
20. Limpeza de contas de teste no banco, criação de contas órfãs corrigida manualmente (usuários que tentaram se cadastrar durante períodos de bug e ficaram sem perfil).

Padrão notável nesta timeline: duas funcionalidades (selfie obrigatória, referências públicas) foram implementadas e depois revertidas na sequência imediata, a pedido explícito do usuário após reconsiderar a decisão de produto. Isso não é bug — é iteração normal de produto, mas quem assumir o projeto deve estar ciente de que o código reflete a **última** decisão, não necessariamente a mais “completa” tecnicamente.

16. Pendências

Crítico

- **Agendar** `publicar_avaliacoes_vencidas()` — sem isso, avaliação unilateral nunca publica após 14

dias. Precisa de `pg_cron` ou função agendada externa.

- **Confirmar status da confirmação de e-mail** (ligada ou desligada) e testar o fluxo completo de ponta a ponta antes de qualquer campanha de aquisição maior.
- **Revisão jurídica formal** dos Termos de Uso, Política de Privacidade e demais textos legais — tudo foi escrito com base em boas práticas, não por advogado.

Importante

- Mecanismo de re-aceite obrigatório quando a versão dos termos muda.
- Registro de IP no consentimento (se for exigência real do negócio).
- Trigger de restrição de coluna em `bookings` (impedir alteração de `valor_combinado` após criado).
- Dashboard de métricas no painel admin.
- Decidir e implementar rotina de exclusão de documentos de verificação (se for reintroduzir essa promessa legal).

Melhoria futura

- Tela de favoritos (schema já existe).
- Marcar mensagens de chat como lidas.
- Busca por turno hoje baixa a tabela `disponibilidade` inteira — reescrever com filtro no banco quando o volume crescer.
- Considerar reativar antecedentes como exigência de documento (hoje é só autodeclaração) quando o volume justificar o esforço operacional.
- Considerar automatizar WhatsApp Business (Meta Cloud API) se o volume de contratações justificar — foi abandonado por complexidade, não por ser tecnicamente inviável.

17. Bugs Conhecidos

Corrigidos (documentados para não reintroduzir)

- **Selects aninhados N:N do PostgREST falhavam silenciosamente** — causa raiz de vários bugs de “dado não aparece”. Solução: sempre consultas separadas.
- **RLS desligado com grant de escrita** em 4 tabelas (auditoria) — qualquer logado editava dados de qualquer profissional.
- **View de feedback privado com `security_invoker=true`** nunca retornava linhas — funcionalidade inoperante até correção.
- `auth_role()` **sem `security_definer`** — função ficava vazia dentro de policies, admin não era

reconhecido.

- **Coluna `visivel` alterada sem recriar a policy dependente** — Postgres recusa `drop column`; correção exige `drop policy` → `alter column` → `create policy` na mesma migração.
- **Corrida entre `signUp` / `signInWithPassword` e leitura assíncrona de `perfil`** — causou login não redirecionando e cadastro mostrando “conta não encontrada” logo após sucesso. Corrigido com padrão de reagir a mudança de estado via `useEffect`, nunca prever timing do SDK.
- **Trigger de sincronização de verificação ausente** — aprovação exigia dois updates manuais do app, e isso já dessincronizou em produção real (identidade aprovada em `verificacoes`, mas `profissionais` continuava “pendente”).
- **`decidirVerificacao` mapeava coluna por ternário** (`tipo === 'identidade' ? ... : antecedentes_status`) — ao adicionar selfie, isso teria gravado aprovação de selfie como aprovação de antecedentes. Corrigido para mapa explícito.

Potencialmente ainda presentes (não confirmado neste handoff)

- **`bookings` sem restrição de coluna no UPDATE** — qualquer uma das partes pode alterar `valor_combinado` depois de criado.
- **Sem retry em falha de envio de e-mail/WhatsApp** — se a Resend estiver fora do ar no momento exato, a notificação se perde (fica registrada como falha em `notificacoes_log`, mas ninguém é avisado ativamente).
- **Sem mecanismo de re-aceite de termos** — se o texto mudar, usuários antigos continuam com acesso mesmo tendo aceitado uma versão desatualizada.

18. Melhorias Sugeridas

- Automatizar a exclusão de documentos de verificação após aprovação, ou remover qualquer expectativa disso do texto legal (já removida) e ser explícito sobre o prazo de retenção real.
- Adicionar testes automatizados — hoje não há nenhum teste no projeto; toda validação foi manual, via uso real do app.
- Considerar um painel de observabilidade simples (mesmo que só uma query SQL salva) para acompanhar `notificacoes_log` e detectar falhas de envio sem precisar que o usuário reclame.
- Avaliar mover a lógica de “completude de perfil” e “selo” para serem consultáveis via uma view só de leitura, facilitando futura exposição em um dashboard admin.

19. Refatorações Recomendadas

- `src/pages/PainelProfissional.jsx` é o maior arquivo do projeto e contém tanto a página

principal quanto o subcomponente `Verificacao` e `Doc`. Vale extrair esses subcomponentes para arquivos próprios em `src/components/`.

- `src/lib/api.js` ultrapassou 1000 linhas. Ainda é gerenciável, mas se crescer mais, vale dividir por domínio (`api/bookings.js`, `api/verificacao.js`, `api/referencias.js`, etc.), mantendo um único ponto de import (`api/index.js`) para não quebrar todos os componentes de uma vez.
 - **Padronizar tratamento de erro:** hoje a maioria das funções em `api.js` faz `if (error) throw error`, mas o tratamento no frontend varia entre `try/catch` com mensagem genérica e alguns lugares mais específicos. Vale um padrão único de exibição de erro.
 - **Extrair estilos inline repetidos** (várias telas repetem os mesmos objetos de `style={{ ... }}` para cards, botões, badges) para classes utilitárias em `global.css`.
-

20. Checklist para Produção

Segurança

- Revisar todas as policies RLS uma última vez antes do lançamento público (rodar a query de auditoria: tabelas com RLS ligado e sem nenhuma policy de SELECT).
- Confirmar que TRUNCATE/REFERENCES/TRIGGER continuam revogados de anon/authenticated.
- Testar fluxo de exclusão de conta de ponta a ponta.

LGPD

- Revisão jurídica formal dos textos legais.
- Decidir sobre registro de IP no consentimento.
- Implementar mecanismo de re-aceite de termos.
- Decidir política real de retenção/exclusão de documentos de verificação.

Testes

- Testar cadastro completo (cliente e profissional) com confirmação de e-mail ligada.
- Testar recuperação de senha de ponta a ponta.
- Testar fluxo de contratação com e sem referência aprovada (disclaimer).
- Testar avaliação dupla simultânea com os 14 dias de prazo (ou mockar a data).

SEO

- Verificar meta tags, Open Graph, sitemap.xml, robots.txt.

Desempenho

- Rodar Lighthouse em conexão 3G simulada (público-alvo usa celular, possivelmente com conexão instável).
- Confirmar que o code splitting por rota está funcionando (bundle inicial pequeno).

Acessibilidade

- Revisar contraste de cores (paleta verde/coral em fundo creme).
- Testar navegação por teclado nos modais (disclaimer, selo).

Deploy

- Confirmar domínio principal (`zeloemcasa.com.br`) com HTTPS ativo.
- Confirmar redirecionamento do domínio antigo (`.netlify.app`).

Monitoramento

- Configurar algum alerta para falhas em `notificacoes_log` (hoje só é visível via SQL manual).

Backups

- Confirmar que o Supabase tem backup automático ativo (verificar plano).

Logs

- Revisar Auth Logs periodicamente por tentativas de acesso suspeitas.

Analytics

- Não há analytics implementado. Decidir se será adicionado, e se for, respeitar a Política de Privacidade (que hoje declara só cookies essenciais).

E-mails

- Confirmar que confirmação de e-mail está no estado desejado (ligada/desligada).
- Testar todos os e-mails transacionais (boas-vindas, aprovação, referência, pedido novo) com o domínio verificado.

Domínio

- `zeloemcasa.com.br` registrado e ativo.

Publicação

- Agendar `publicar_avaliacoes_vencidas()` .
 - Recrutar e cadastrar profissionais reais suficientes para a busca não ficar vazia no lançamento.
-

21. Resumo Executivo

Percentual estimado de conclusão

~75-80% do MVP funcional. O núcleo do produto (cadastro, verificação, busca, contratação, avaliação, referências, LGPD) está implementado e testado manualmente de ponta a ponta. O que falta é majoritariamente operacional (agendar cron job, revisão jurídica) e polimento (dashboard admin, testes automatizados).

Qualidade da arquitetura: 7/10

Pontos fortes: separação clara de responsabilidades (`api.js` como única porta pro banco), uso consistente de RLS como camada real de segurança (não só decorativa), triggers no banco para invariantes de negócio (pontuação, selo, sincronização) em vez de confiar no frontend.

Pontos fracos: nenhuma automação de teste; algumas migrações corrigem a mesma coisa duas vezes (sinal de que decisões foram tomadas e revertidas rápido, o que é normal em produto jovem, mas deixa rastro de complexidade acumulada — 27 migrações para um schema que poderia ter nascido mais enxuto se todas as decisões fossem conhecidas de início, o que nunca é o caso na prática).

Qualidade do código: 7/10

Comentários extensos e úteis, específicos sobre *por que* uma decisão foi tomada (não só o *que* o código faz) — isso é incomum e valioso para handoff. Como contraponto, alguns arquivos cresceram demais (`PainelProfissional.jsx` , `api.js`) e mereceriam divisão antes de crescerem mais.

Riscos técnicos

1. **Falta de testes automatizados** — qualquer alteração futura depende de teste manual repetido, o que é lento e propenso a deixar passar regressão (como as próprias reversões de funcionalidade nesta timeline mostram).
2. `publicar_avaliacoes_vencidas()` **não agendada** — funcionalidade de avaliação incompleta sem isso, e é fácil esquecer porque não gera erro visível, só um silêncio (avaliação unilateral nunca é publicada).
3. **Dependência de configuração manual em três Edge Functions** — qualquer republicação futura corre risco de esquecer de desligar "Verify JWT", o que já causou confusão real durante o desenvolvimento.

Prioridades para os próximos passos

1. Agendar a publicação de avaliações vencidas (crítico, rápido de resolver).
2. Confirmar e testar o estado real da confirmação de e-mail.
3. Buscar revisão jurídica formal antes de qualquer campanha de aquisição em escala.
4. Recrutar profissionais reais suficientes — o produto tecnicamente funciona, mas só cria valor com oferta real na busca.

5. Só depois disso, investir em refatoração de arquivos grandes e testes automatizados — não é urgente para o lançamento, mas vai cobrar seu preço conforme o time crescer.

Fim do documento. Este handoff foi escrito a partir do histórico de uma conversa de desenvolvimento incremental, não de uma auditoria fresca de código nesta sessão — pontos marcados como “não confirmado neste handoff” merecem verificação direta no repositório e no painel do Supabase antes de decisões importantes.